



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 4: Công nghệ thành phần với .NET

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Nội dung của chương này tập trung xem xét một số chủ đề sau đây:

- .NET Framework, một số khái niệm của các thành phần .NET.
- Các kiểu thành phần .NET, kết nối giữa các thành phần, và cách triển khai chúng.
- Các thành phần cục bộ và phân tán, các thành phần kết hợp và hợp thành.
- Phương thức đồng bộ và không đồng bộ.
- Hướng dẫn từng bước để xây dựng, triển khai, và sử dụng các thành phần .NET

| 4.1 GIỚI THIỆU .NET FRAMEWORK



4.1.1 Tổng quan về .NET:

- .NET là một trong những công nghệ nổi tiếng của Microsoft được giới thiệu vào năm 2000 giúp cho việc xây dựng, triển khai nhanh chóng các ứng dụng.
- .NET dựa trên thành phần, cung cấp một môi trường đa ngôn ngữ, hiệu năng cao cho việc phát triển ứng dụng, lõi của .NET bao gồm các file XML và giao thức truy nhập đối tượng đơn giản (SOAP: Simple Object Access Protocol) để cung cấp dịch vụ web thông qua Internet.
- Mục đích của .NET là để thuận tiện cho việc phát triển các ứng dụng máy để bàn và các dịch vụ ứng dụng dựa trên nền Web, làm cho dịch vụ như luôn sẵn sàng và có thể truy nhập được không chỉ trên nền Windows mà còn trên các nền tảng khác thông qua các giao thức phổ biến như SOAP và HTTP.

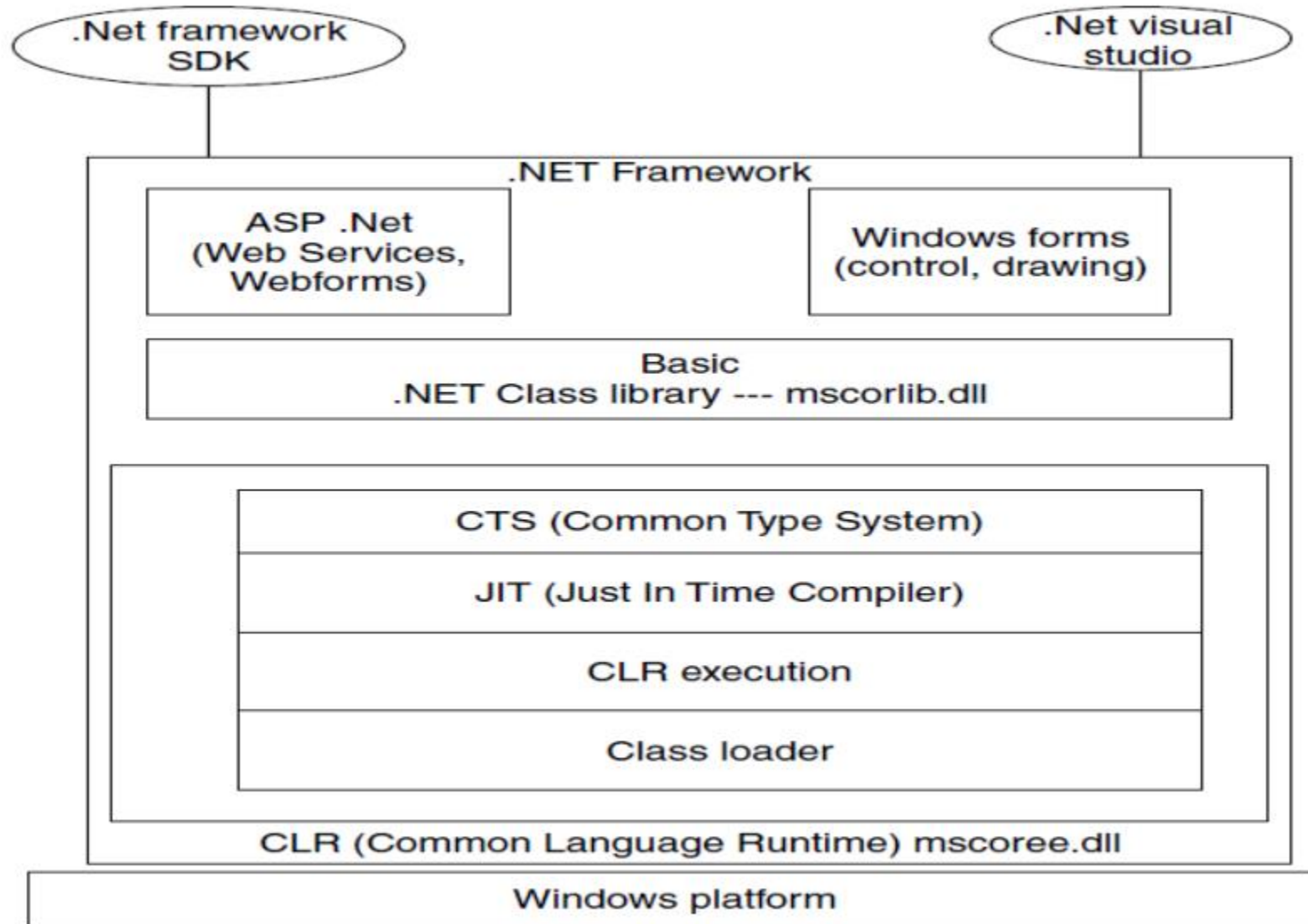
| Một số đặc trưng của .NET

- Làm đơn giản việc thành phần hóa với công nghệ thành phần COM (Object Model) và công nghệ phân tán DCOM. Về nguyên lý các thành phần COM có thể sử dụng lại trong việc xây dựng thành phần phần mềm và ứng dụng, tuy nhiên, tiến trình phát triển cũng rất phức tạp.
- Cho phép triển khai thành phần theo cách lắp ráp, điều này cho phép nhiều phiên bản của các thành phần cùng tên có thể cùng tồn tại mà không có bất kỳ xung đột nào.
- Giúp dễ dàng phát triển các thành phần phân tán bằng công nghệ truyền thông từ xa. .NET framework hỗ trợ khả năng phối hợp hoạt động giữa các phần giữa COM và các thành phần .NET. Việc phát triển các thành phần trong .NET dễ dàng hơn là trong COM.

| Một số đặc trưng của .NET

- Web Service thay thế công nghệ MS DCOM cho các ứng dụng Internet được hỗ trợ bởi các giao thức XML, SOAP, và HTML.
- .NET khắc phục việc thiếu hỗ trợ tương lửa của DCOM và làm cho các dịch vụ trở nên sẵn sàng giữa các platform thông qua các giao thức gắn kết lỏng lẻo XML và SOAP.
- .NET framework có sẵn trong SDK và Visual Studio .NET IDE SDK.
- .NET framework bao gồm 2 phần chính: Common Language Runtime (CLR) và một tập thống nhất các thư viện để xây dựng các ứng dụng Web, Windows Forms.
- Hỗ trợ tất cả các ngôn ngữ .NET như VB.NET, C#, F# ..., và các ngôn ngữ khác trong bộ .NET SDK.

| Kiến trúc của .NET framework



| Ưu điểm của .NET Framework

- Đảm bảo tính tương thích với các hệ điều hành Windows.
- Hỗ trợ cho việc phát triển các ứng dụng Windows, Web, Mobile và Cloud.
- Hỗ trợ cho nhiều ngôn ngữ lập trình.
- Cung cấp rất nhiều thư viện và công cụ hỗ trợ việc phát triển.
- Tối ưu hóa hiệu suất và tốc độ của ứng dụng.
- Quản lý và bảo mật dữ liệu tốt.

| Nhược điểm của .NET Framework

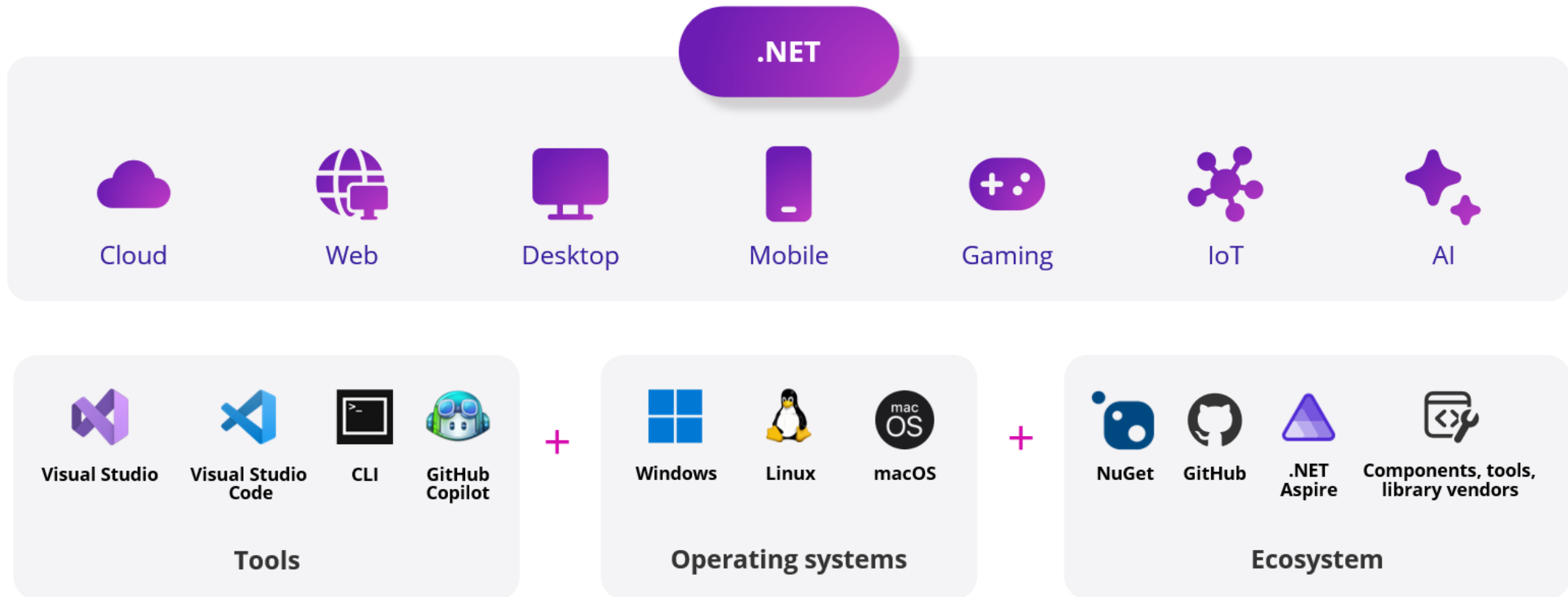
- Yêu cầu hệ điều hành Windows để chạy, người dùng cũng chỉ chạy được những chương trình thiết kế dựa trên .Net Framework khi máy tính đã cài đặt .Net Framework.
- Kích thước của nó rất lớn, có thể gây trễ cho việc cài đặt và khởi chạy.
- Không hoạt động tốt trên các thiết bị có cấu hình yếu.

| Giới thiệu .NET Core

- .NET core là một Framework được phát hành bởi Microsoft vào năm 2016 để xây dựng ứng dụng đa nền tảng.
- Có thể xây dựng ứng dụng Desktop (WPF, Winform), Website, Mobile, Game, IOT và cả AI.
- Các phiên bản của .NET Core: từ .NET 5 (2020) Microsoft đã bỏ chữ core và gọi chung là .NET, phiên bản mới T2/2025 nhất là .NET 9.0.
- Đặc điểm:
 - Đa nền tảng: Chạy trên các hệ điều hành Windows, macOS và Linux.
 - Nhất quán trên các kiến trúc: có thể chạy trên kiến trúc x64, x86 và ARM.
 - Công cụ dòng lệnh dễ sử dụng.
 - Triển khai linh hoạt, có thể được sử dụng với các container docker.
 - Nền tảng là nguồn mở, được hỗ trợ bởi Microsoft.
 - .NET core nhanh và nhẹ hơn .NET Framework: không cần tải toàn bộ Framework, chỉ cần cài bộ SDK và khi cần package nào thì tải xuống nên nhẹ hơn.

| Giới thiệu .NET

Build anything with a unified platform



.NET 9

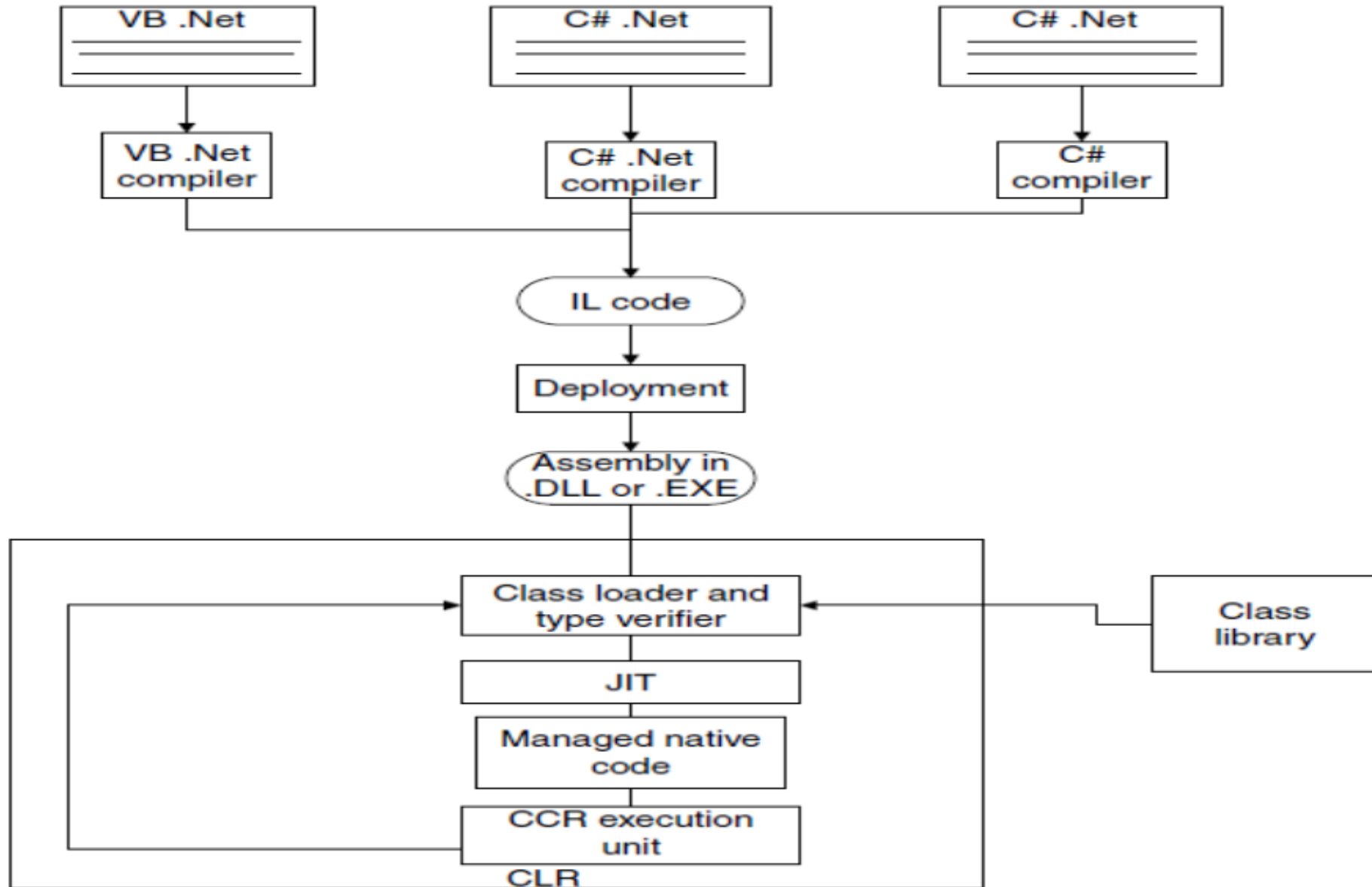
| 4.1.2 Common Language Runtime (CLR)

- CLR là môi trường thực thi cho các ứng dụng .NET. Nó quản lý việc biên dịch, quản lý bộ nhớ, xử lý ngoại lệ và quản lý luồng của ứng dụng. CLR cung cấp môi trường chung để chạy các ứng dụng .NET.
- CLR là một môi trường máy ảo nằm trên đỉnh của hệ điều hành Windows giống JVM trong Java. CLR bao gồm Common Type System (CTS), Just-In-Time Compiler (JIT), Execution unit (đơn vị thực thi), cùng với các dịch vụ quản lý khác như kết nối dữ liệu và quản lý bảo mật.
- Tất cả các thành phần phần mềm được tập hợp lại trong một gói assembly (trong kiến trúc Java là file .jar) bao gồm mã MS Intermediate Language (MSIL) và file manifest (metadata miêu tả về gói này).
- Mã IL được biên dịch thành mã máy bởi trình biên dịch JIT. Mã IL được kiểm tra lại bởi CTS để kiểm tra tính hợp lệ của kiểu dữ liệu sử dụng trong mã đó.

| Common Language Runtime (CLR)

- Mã IL giống như mã byte code của Java, nó có thể giao tiếp với bất kỳ loại mã nào thông qua sự hỗ trợ của CLR. Mã IL có thể có định dạng là tập tin thực thi (.EXE) hoặc thư viện liên kết động kiểu (.DLL).
- Nếu mã IL này được tạo ra bởi trình biên dịch .NET, chúng được gọi là mã có quản lý (managed code) và chỉ được thực thi trên nền tảng .NET. Một số file DLL hoặc EXE không được tạo ra bởi trình biên dịch .NET được gọi là mã không quản lý được.

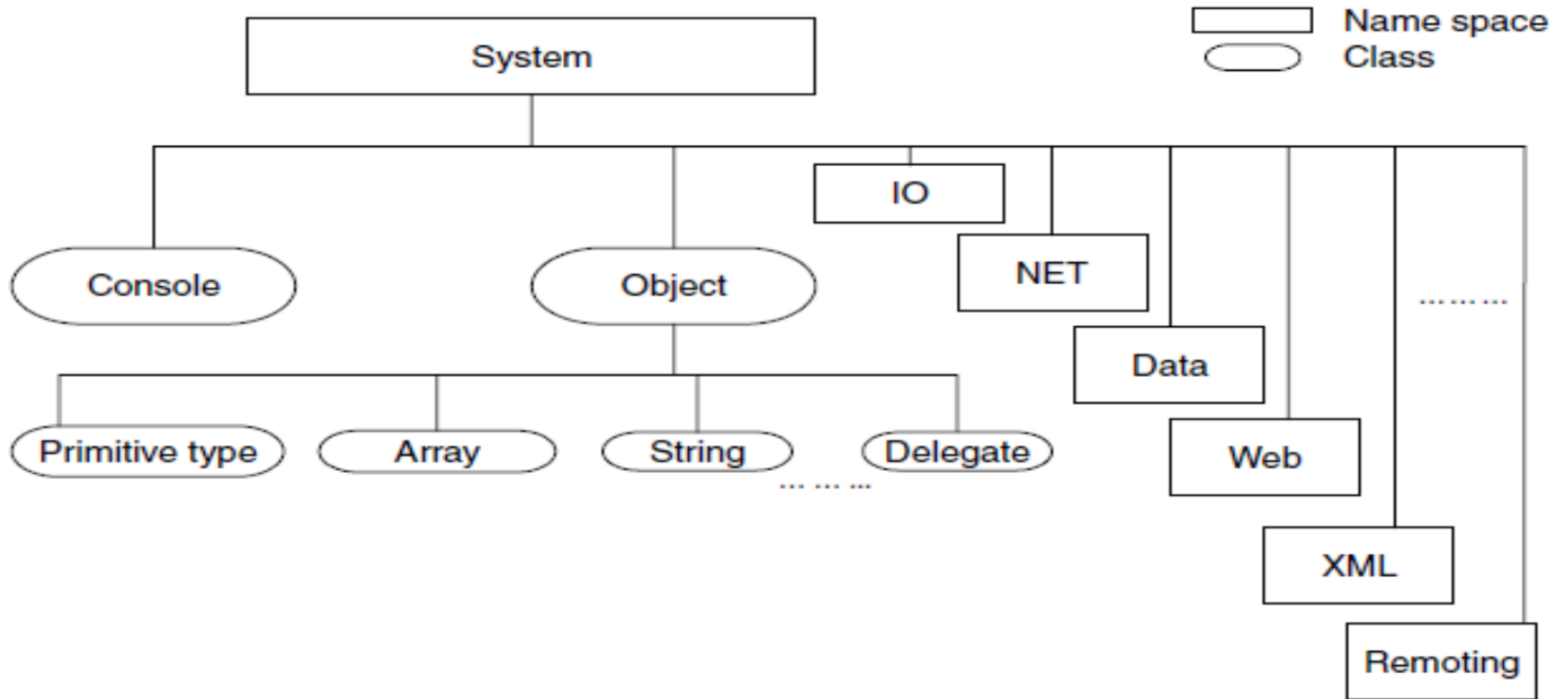
| Common Language Runtime (CLR)



| Thư viện lớp của .NET

- Thư viện lớp của .NET là một tập hợp các lớp cơ bản có thể sử dụng lại và được tổ chức bởi không gian tên ([namespace](#)).
- Một không gian tên bao gồm nhiều lớp và các không gian tên con. Nó được triển khai như một thư viện lớp thành phần và được tổ chức thành một cây phân cấp dựa trên thành phần.
- Namespace gốc trong thư viện lớp là System, nó bao gồm nhiều lớp cơ bản như Object, Console, và nhiều subnamespace như IO, Net, Data, Remoting...
- Các nhà phát triển có thể tùy chỉnh namespace và tổ chức các lớp có liên quan trong một namespace tùy chỉnh.
- Để sử dụng các lớp trong một namespace, cú pháp [using <namespace>](#) trong C# hoặc cú pháp [import <namespace>](#) trong VB phải được đưa vào ở phần đầu của code. Thư viện lớp built-in cơ bản của hệ thống đã được triển khai trong file [mscorlib.dll](#).

| Thư viện lớp của .NET



| 4.2 MÔ HÌNH THÀNH PHẦN CỦA .NET

- Công nghệ thành phần .NET tăng cường và đơn giản hoá sự tồn tại của các công nghệ MS COM, DCOM, COM+. Các thành phần .NET dễ dàng phát triển hơn COM và DCOM. Chúng giải quyết vấn đề xung đột version **DLL Hell** và vấn đề firewall trong DCOM.
- NET framework được xây dựng theo mô hình thành phần. Bất kì thành phần .NET nào cũng có định dạng của MSIL đã được biên dịch trước đó.
Ví dụ: System namespace System.Runtime.Remoting đã có sẵn trong mscolib.dll và System.XML namespace đã có sẵn trong System.XML.dll.
- Một file **.dll** là một thành phần đã được triển khai của .NET (Assembly).

I MÔ HÌNH THÀNH PHẦN CỦA .NET

- Một thành phần .NET là một module MSIL đơn được biên dịch trước và xây dựng từ một hoặc nhiều class hoặc nhiều module được triển khai trong một assembly file DLL.
- Một assembly chứa bốn phần:
 - Bảng ghi thông tin (Manifest) gồm tên assembly, phiên bản...
 - Siêu dữ liệu (Metadata) của module.
 - Mã IL của module.
 - Các tài nguyên như các file ảnh...
- Module bao gồm mã MSIL và siêu dữ liệu nhưng không có manifest, nó có phần mở rộng là .netmodule.
 - Có thể có một hoặc nhiều class trong một module.
 - Một assembly được tạo ra từ một hoặc nhiều class trong một module.
 - Mỗi module có thể được code bằng nhiều ngôn ngữ khác nhau nhưng cuối cùng phải có cùng định dạng MSIL.
 - Một file .dll là một file mã nhị phân không có khả năng thực thi, giống như file .class trong Java.

| Phân biệt Assembly và Module

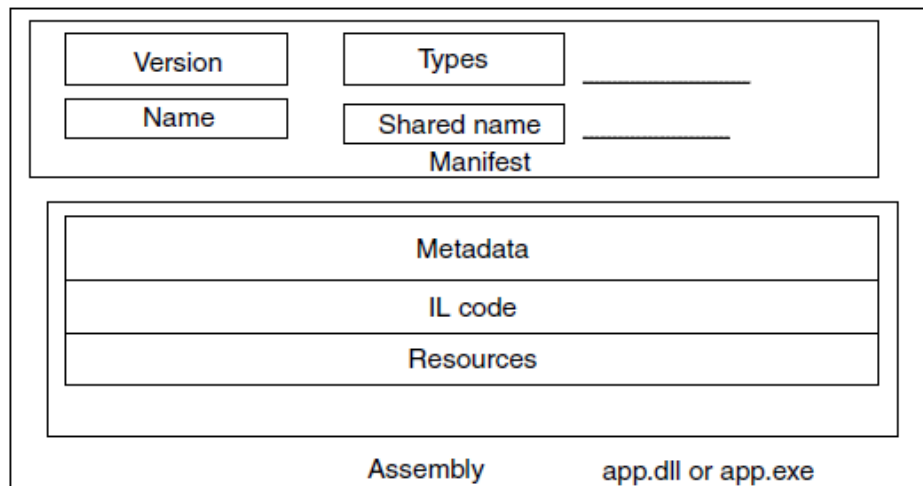
- Assembly

- Assembly là đơn vị triển khai tối thiểu trong .NET, nó là 1 dạng của EXE hoặc DLL.
- Một assembly có thể chứa 1 hoặc nhiều files, chúng có thể là các file resource hoặc module.
- Một assembly luôn luôn có 1 file manifest, nó là siêu dữ liệu của assembly.
- Có thể xem cấu trúc của assembly bằng công cụ IL Disassembler (ildasm.exe).

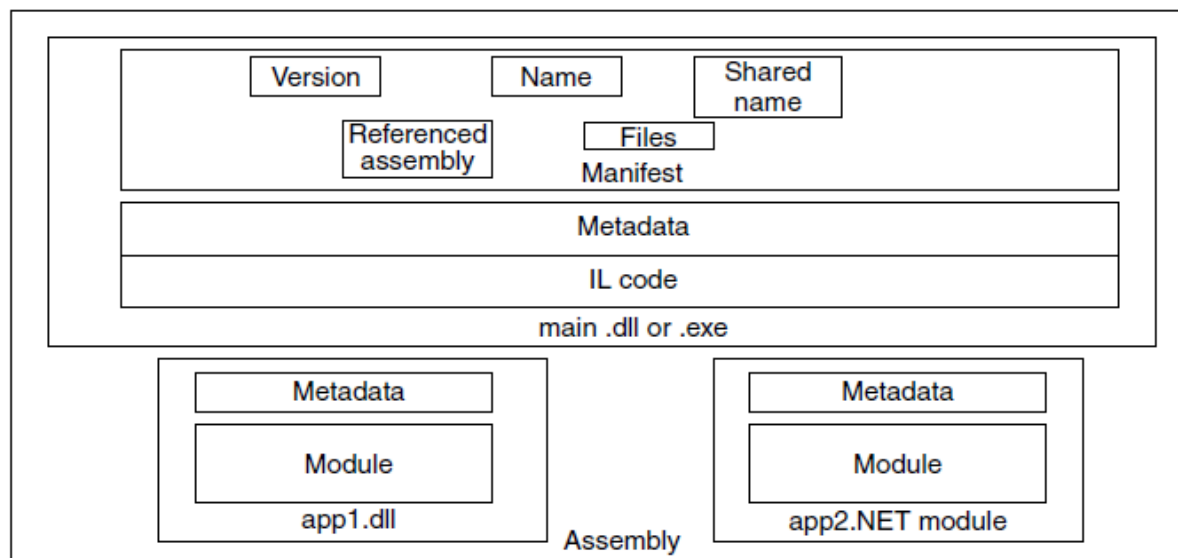
- Module

- Các file code trong một assembly được gọi là các module.
- Một module là đơn vị tối thiểu của biên dịch.
- Module chứa metadata nhưng không có manifest.
- Module không thể triển khai độc lập, nó phải được liên kết trong assembly.

| Cấu trúc file assembly



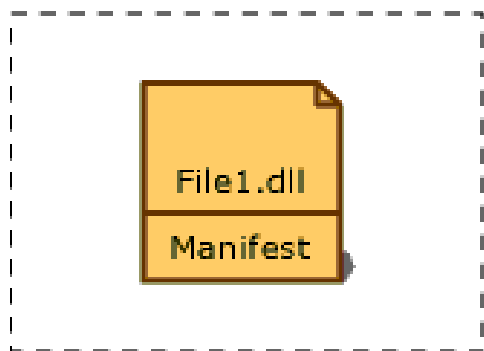
(a) Single file assembly



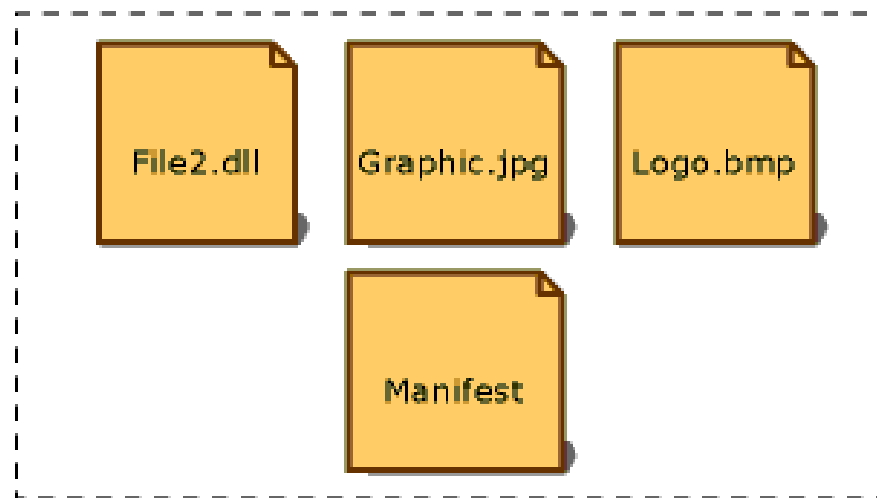
(b) Multiple file assembly

| Cấu trúc file assembly

A single-file assembly



A multifile assembly



Ví dụ file assembly

| Case study

```
using System;
namespace TempConv
{
    public class TempConvComp
    {
        public double cToF(double c)
        {
            return (int)((c * 9) / 5.0 + 32);
        }
        public double fToC(double f)
        {
            return (int)((f - 32) * 5 / 9.0);
        }
    }
}
```

TempConvComp.cs

- Xây dựng module:

>csc /t:module TempConvComp.cs ->
TempConvComp.netModule

- Thêm module vào một thành phần:

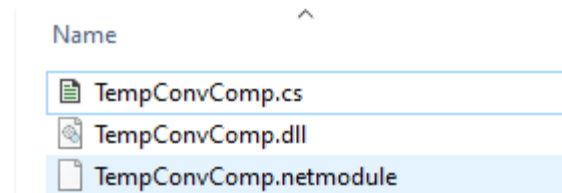
>csc /t:library /addmodule:
TempConvComp.netmodule
anotherComp.dll

- Xây dựng DLL thành phần:

>csc /t:library TempConvComp.cs ->
TempConvComp.dll

Lưu ý:

- Add đường dẫn sau vào biến PATH trong Environment Variables để thực thi câu lệnh **csc**:
C:\Windows\Microsoft.NET\Framework64\v4.0.30319
(hoặc đường dẫn .NET tương ứng trên máy tính)



| Case study

Sử dụng lại thành phần **TempConvComp** đã xây dựng trước đó với C#.

```
using System;
using TempConv;
namespace TempConvCSClient
{
    class MainApp
    {
        public static void Main()
        {
            TempConv.TempConvComp myCSTempConvComp = new TempConv.TempConvComp();
            double choice, input, output;
            bool next = true;
            while (next)
            {
                Console.WriteLine("Please enter your choice:");
                Console.WriteLine("\t\t 1 - Converter from F to C");
                Console.WriteLine("\t\t 2 - From C to F");
                Console.WriteLine("\t\t 3 - exit");
                choice = Double.Parse(Console.ReadLine());
                if (choice == 1)
                {
                    Console.WriteLine("Please tell me the temperature in F:");
                    input = Double.Parse(Console.ReadLine());
                    output = myCSTempConvComp.fToC(input);
                    Console.WriteLine("result = {0} ", output);
                }
                else if (choice == 2)
                {
                    Console.WriteLine("Please tell me the temperature in C:");
                    input = Double.Parse(Console.ReadLine());
                    output = myCSTempConvComp.cToF(input);
                    Console.WriteLine("result = {0}", output);
                }
                else
                {
                    next = false;
                    Console.WriteLine("see you next time");
                }
            }
        }
    }
}
```

- Xây dựng ứng dụng:

>csc /t:exe /r:TempConvTemp.dll
TempConvCSClient.cs -> TempConvCSClient.exe

- Kết quả:

Name
TempConvComp.cs
TempConvComp.dll
TempConvCSClient.cs
TempConvCSClient.exe

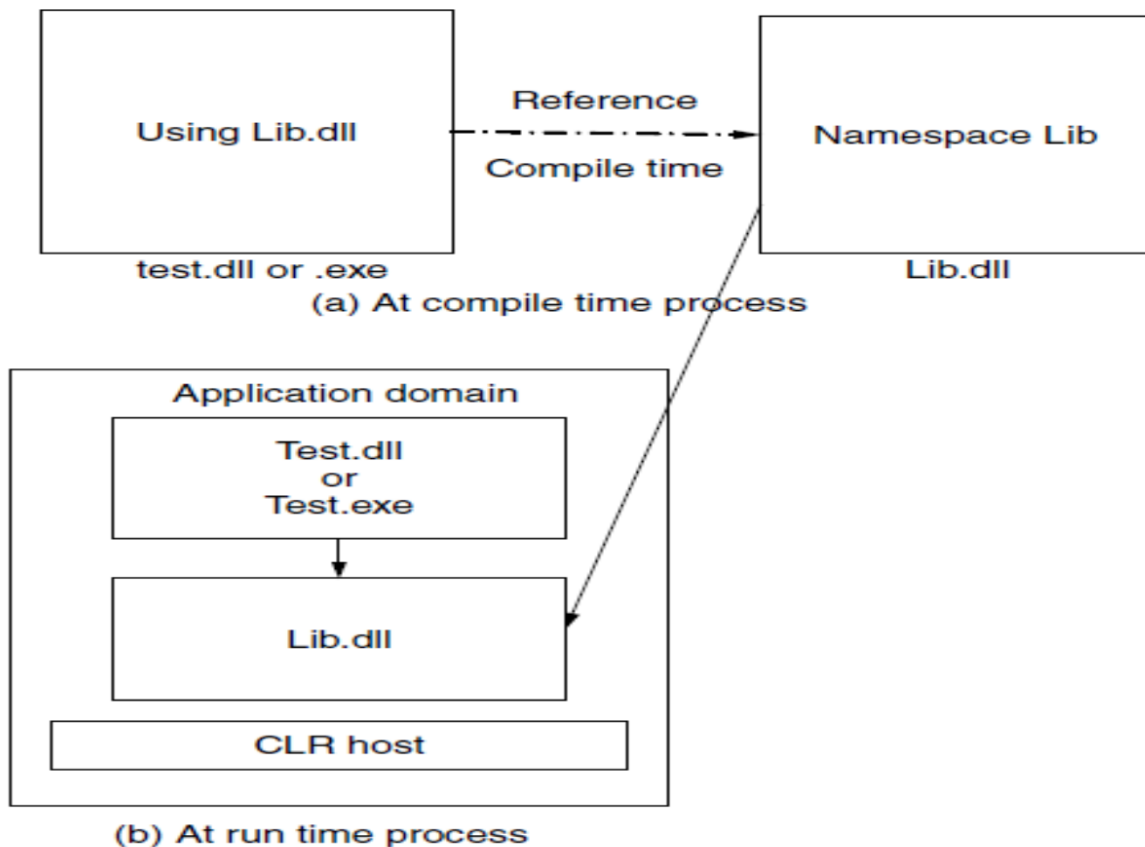
```
Please enter your choice:
        1 - Converter from F to C
        2 - From C to F
        3 - exit
1
Please tell me the temperature in F:
90
result = 32
Please enter your choice:
        1 - Converter from F to C
        2 - From C to F
        3 - exit
2
Please tell me the temperature in C:
24
result = 75
Please enter your choice:
        1 - Converter from F to C
        2 - From C to F
        3 - exit
```

| Case study

Sử dụng lại thành phần **TempConvComp** đã xây dựng trước đó, tương tự với ngôn ngữ C++.

```
#include "stdafx.h"
#include <iostream>
#include <mscorlib.dll>
#include "TempConv.dll"
using namespace System;
using namespace std;
#ifdef _UNICODE
int wmain(void)
#else
int main(void)
#endif
{
    int choice;
    double input;
    double output;
    bool next = true;
    TempConv::TempConvComp *myCSConvComp = new TempConv::TempConvComp();
    while (next) {
        Console::WriteLine("Please enter your choice: ");
        Console::WriteLine("\t\t1 - Converter from F to C");
        Console::WriteLine("\t\t2 - Converter from C to F");
        cin >> choice;
        if (choice == 1) {
            Console::WriteLine("Please input the temperature in F : ");
            cin >> input;
            output = myCSConvComp->fToC(input);
            Console::WriteLine(output);
        } else if (choice == 2) {
            Console::WriteLine("Please input the temperature in C : ");
            cin >> input;
            output = myCSConvComp->cToF(input);
            Console::WriteLine(output);
        } else {
            next = false;
            Console::WriteLine("See you next time");
        }
    }
    return 0;
}
```

| Các thành phần .NET tại thời điểm chạy và thời điểm biên dịch

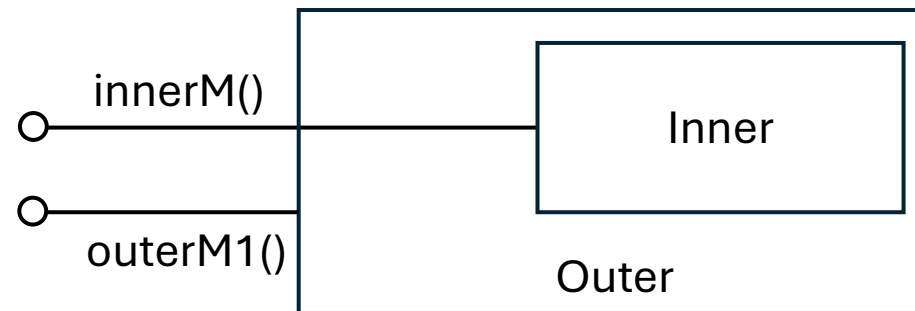


- Client của một thành phần tạo một tham chiếu đến thành phần server tại thời điểm biên dịch
- Và sau đó client tải thành phần tự động vào miền ứng dụng của nó tại thời gian chạy khi nó cần.

| 4.3 MÔ HÌNH KẾT NỐI CỦA .NET

4.3.1 Thành phần kết nối .NET:

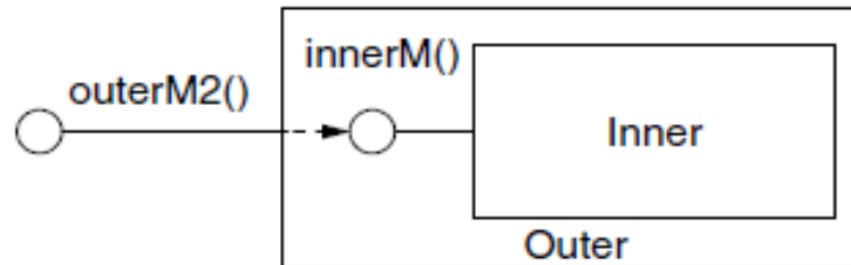
- Tổ hợp liên kết:
 - Dịch vụ của thành phần bên trong sẽ cung cấp dịch vụ trực tiếp cho thành phần client bên ngoài.
 - Phương thức *innerM()* của thành phần bên trong trở thành một phần của giao diện với thành phần bên ngoài như hình:



| 4.3 MÔ HÌNH KẾT NỐI CỦA .NET

Thành phần kết nối .NET:

- Tổ hợp bao hàm:
 - Nếu một lời gọi đến thành phần bên ngoài cần sự trợ giúp của thành phần bên trong, lời gọi đó sẽ được chuyển đến thành phần bên trong đó.
 - Thành phần bên ngoài giấu giao diện của thành phần bên trong, Client không thấy được việc chuyển giao lời gọi.
 - Phương thức *outerM()* đẩy một lời gọi đến phương thức *innerM()* của thành phần bên trong như trong hình:

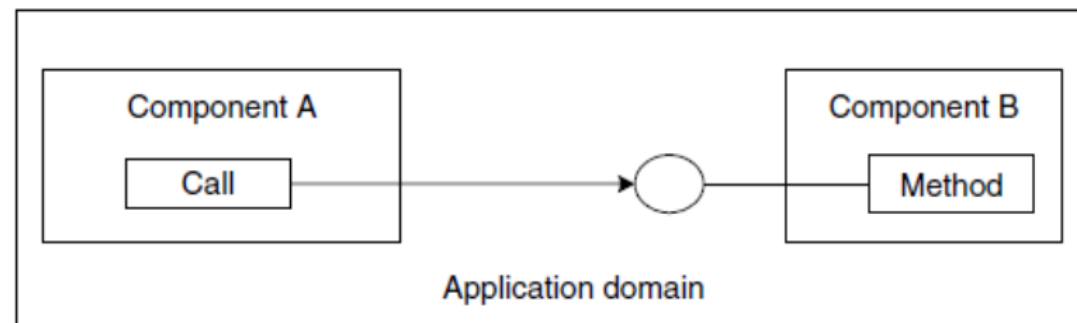


- Một thành phần .NET còn có thể được tạo thành nhờ việc kết hợp **tổ hợp liên kết** và **tổ hợp bao hàm** theo kiểu cấu trúc phẳng hoặc lồng các tổ hợp ở nhiều mức sâu hơn.

Ví dụ

```
using System;
namespace NS1
{
    public class Inner
    {
        public void innerM ()
        {
            Console.WriteLine ("I am Inner.")
        }
    }
}

using System;
using NS1;
public class Outer
{
    public Inner i = new Inner ();
    //aggregation: Outer expose the Inner
    public void outerM1 ()
    {
        Console.WriteLine ("I am outer.");
    }
    public void outerM2() //delegation in containmentment
    {
        i.innerM();
    }
    public static void main()
    {
        outer o1 = new Outer();
        Inner i1 = o1.i;
        i1.innerM(); //interface to the aggregate
        o1.outerM();
        o1.outerM2(); // interface to the containmentment
        Inner i2 = new Inner();
        i2.innerM();
    }
}
```



Gọi trực tiếp một thành phần

Kết hợp các tổ hợp liên kết và bao hàm

| 4.3.2 Kết nối các thành phần bằng Sự kiện và Ủy nhiệm

- **NET Delegate** là một kiểu phương thức tham chiếu đến một phương thức giống như con trỏ hàm trong C++ nhưng nó an toàn và đảm bảo về kiểu.
- Một Delegate sẽ ủy nhiệm luồng điều khiển đến bộ điều khiển sự kiện đã đăng ký của nó khi sự kiện xảy ra. Nó hoạt động như một người quan sát, tương tự như một bộ lắng nghe sự kiện trong Java.
- Một thể hiện của Delegate có thể chứa một phương thức tĩnh của một lớp, hoặc một phương thức của một thành phần, hoặc một phương thức của chính đối tượng là nó. Có hai kiểu Delegate: **SingleCast** và **MultiCast**.

| Kết nối các thành phần bằng Sự kiện và Ủy nhiệm

- Một **SingleCast** chỉ có thể ủy nhiệm một phương thức một lần.

Ví dụ:

- MyDelegate là một Delegate tham chiếu đến bất cứ phương thức nào trả về kiểu int và không có tham biến.
- Các đặc trưng của objMethod và StaticMethod thỏa mãn Delegate MyDelegate. Lệnh dlg() đầu tiên gọi đến objMethod và lệnh dlg() thứ hai gọi đến phương thức StaticMethod().

```
Delegate int Mydelegate();  
public class MyClass  
{  
    public int ObjMethod() { - - - }  
    static public int StaticMethod () { - - - }  
    public class Drive { Static public void Main()  
    {  
        MyClass c = new MyClass();  
        MyDelegate dlg = new MyDelegate(c.objMethod());  
        dlg();  
        dlg = new MyDelegate (MyClass.StaticMethod());  
        dlg();  
    }  
}
```

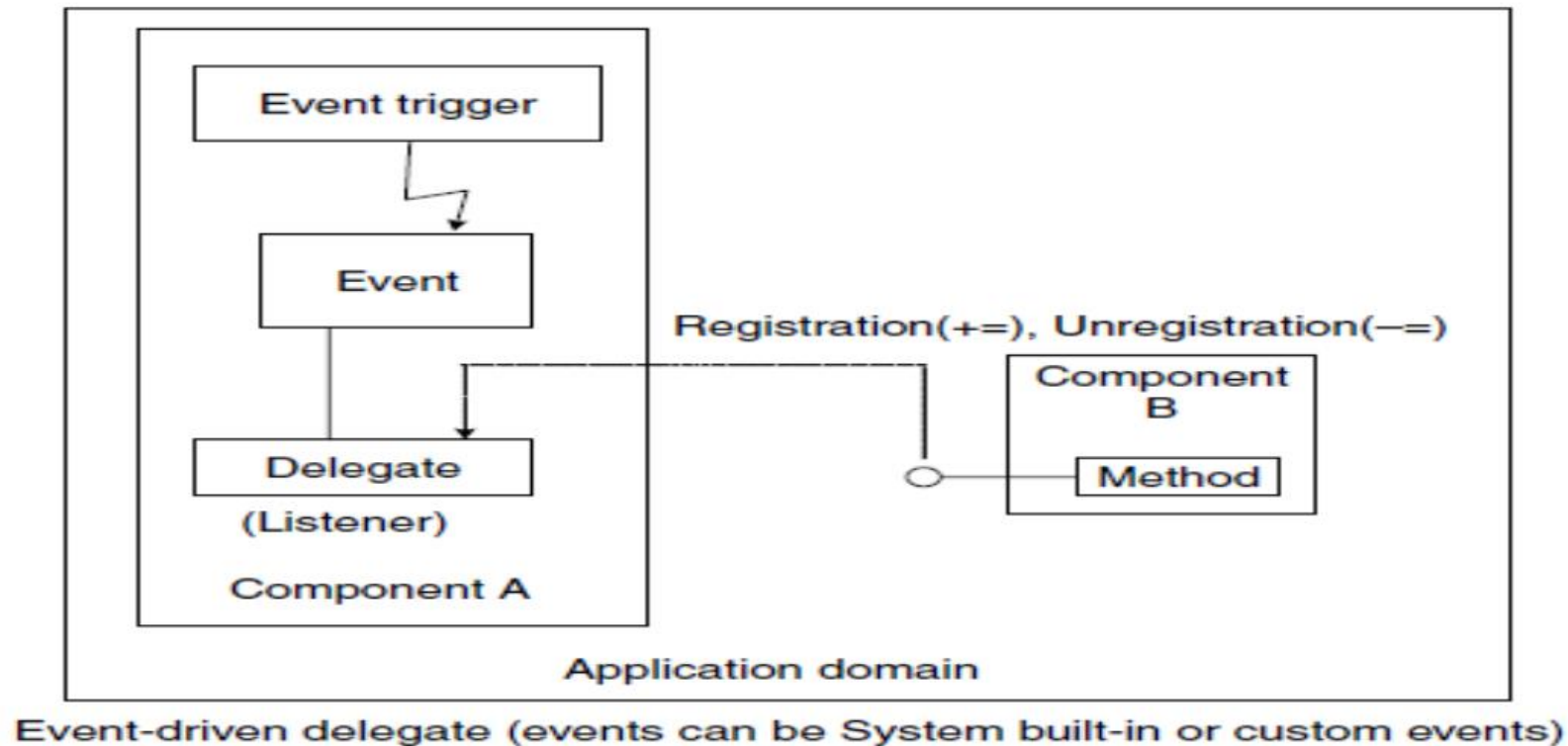
| Kết nối các thành phần bằng Sự kiện và Ủy nhiệm

- **MultiCast Delegate** có kiểu trả về là void và có thể nối với nhiều phương thức, và gọi chúng theo thứ tự đăng ký.
 - Việc đăng ký được thực hiện bởi lệnh `+= Delegate` và việc hủy bỏ đăng ký được thực hiện bởi lệnh `-= Delegate`.

```
Delegate void MultiDelegate();  
MultiDelegate mdlg = null;  
mdlg += new MultiDelegate (Method1);  
mdlg += new MultiDelegate (Method2);
```

| Kết nối các thành phần bằng Sự kiện và Ủy nhiệm

- Delegate đóng vai trò bộ lắng nghe, và bộ xử lý sự kiện phải đăng ký với bộ lắng nghe này để xử lý sự kiện ngay khi sự kiện được kích hoạt.



| Kết nối các thành phần bằng Sự kiện và Ủy nhiệm

- Một **sự kiện** là một thông điệp được một đối tượng gửi đi để gọi một hành động. Đối tượng phát ra sự kiện là nguồn sự kiện, và đối tượng nhận và xử lý sự kiện là đích sự kiện.
-> Đây là một kiểu giao tiếp hướng sự kiện giữa các thành phần hoặc trong cùng một thành phần.
- Lớp Delegate là một lớp kênh giao tiếp giữa nguồn sự kiện và đích sự kiện.

| Thủ tục để tạo và sử dụng sự kiện

1. Tạo một lớp delegate:

```
public class DelegateStart;
```

2. Tạo một lớp chứa miền delegate, lớp MyClass.

```
public event DelegateStart EventStart;
```

```
public class MyClass { - - -}
```

3. Định nghĩa bộ xử lý sự kiện.

```
public void handleEvent() { - - -}
```

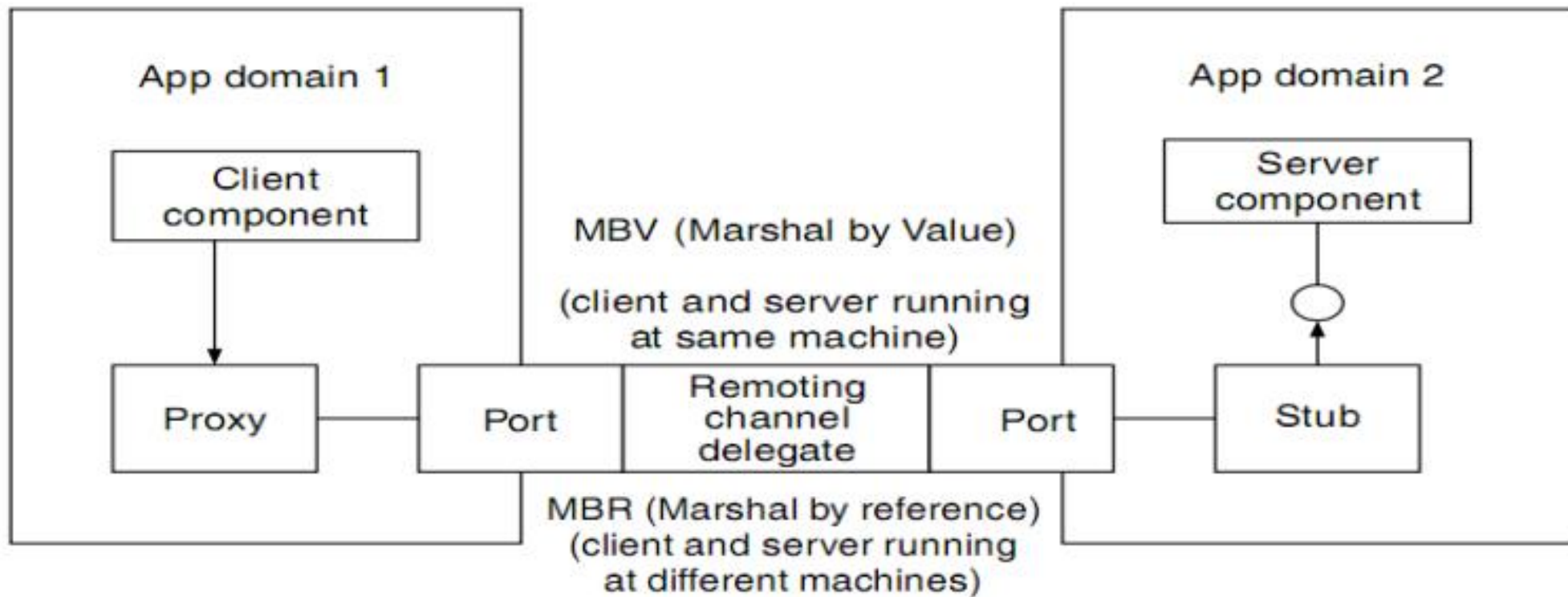
4. Kết nối sự kiện ủy nhiệm với một bộ xử lý sự kiện thông qua bộ lắng nghe sự kiện, kích hoạt một sự kiện, gọi đến bộ xử lý sự kiện.

```
public static void Main() {  
    MyClass EventObj = new MyClass();  
    EventObj.EventStart += new DelegateStart(handleEvent);  
    EventObj.EventStart();  
    ...  
}
```

| 4.3.3 Các bộ kết nối từ xa cho các thành phần phân tán .NET

- Một thành phần hay một client không thể truy nhập trực tiếp tới một thành phần từ xa đang chạy trên một miền ứng dụng khác cũng như các tiến trình khác trừ khi nó sử dụng kênh kết nối từ xa.
- Có hai cách tạo đối tượng để nó có thể gọi một phương thức từ xa của một thành phần phân tán:
 - Theo kiểu **MBV** (Marshal by Value), server truyền bản sao của đối tượng cho client.
 - Theo kiểu **MBR** (Marshal by Reference), client sẽ tạo một tham chiếu của đối tượng từ xa.

| Lời gọi phương thức đồng bộ từ xa của thành phần phân tán



| Ví dụ

Xây dựng thành phần phân tán được truy nhập từ xa:

```
using System;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;
public class CoTempConv : MarshalByRefObject
{
    public static void Main()
    {
        TcpChannel channel = new TcpChannel(4000);
        ChannelServices.RegisterChannel(channel);
        RemotingConfiguration.
            RegisterWellKnownServiceType (typeof(CoTempConv), "TempConvCompDotNet",
                WellKnownObjectMode.Singleton);
        System.Console.WriteLine("Hit <enter> to exit...");
        System.Console.ReadLine();
    }
    public double cToF(double c)
    {
        return (int) ((c*9/5.0+32)*100)/100.0;
    }
    public double fToC(double f)
    {
        return (int) ((f-32)*5/9.0*100)/100.0;
    }
}
```

CoTempConv.cs

| Ví dụ

Xây dựng Client:

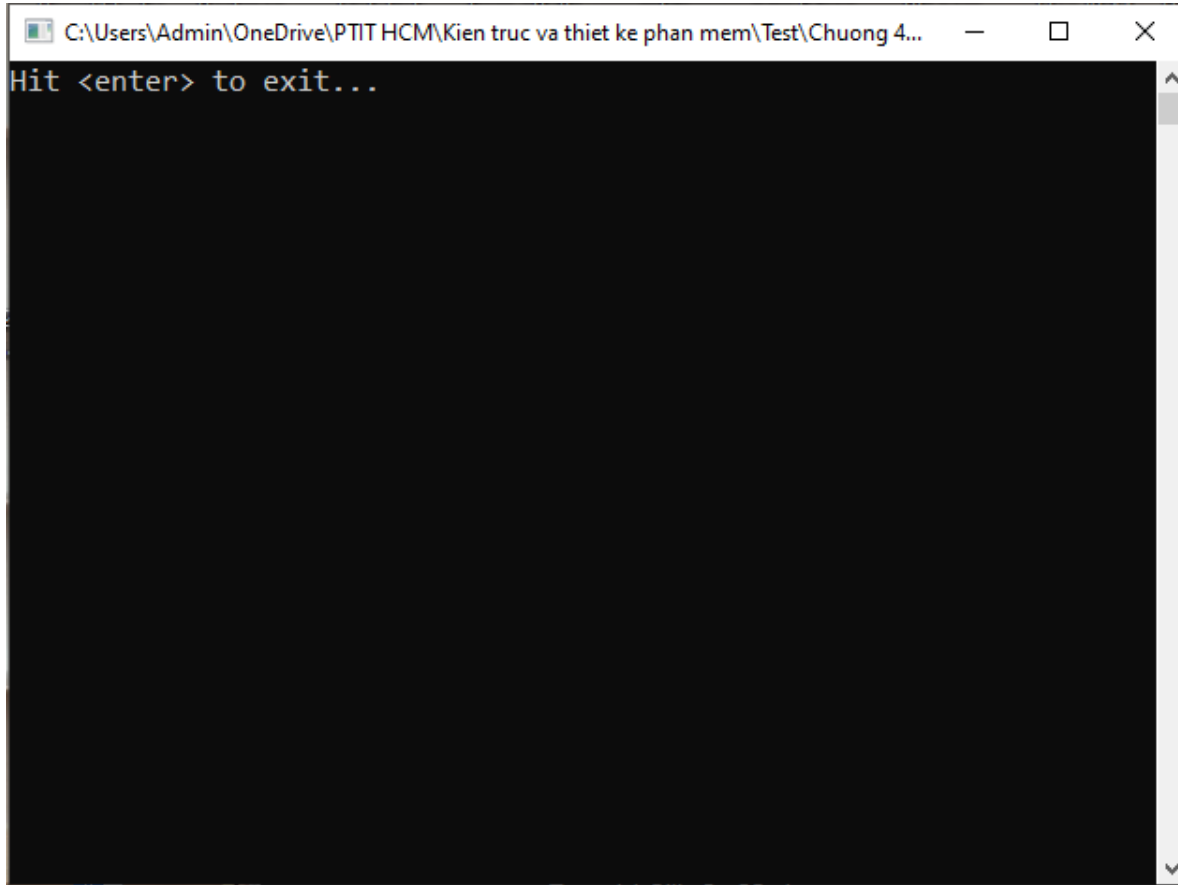
```
using System.Runtime.Remoting;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;
class CoTempClient
{
    public static void Main()
    {
        try
        {
            TcpChannel channel = new TcpChannel();
            ChannelServices.RegisterChannel(channel);
            CoTempConv myCSTempConvComp = (CoTempConv)Activator.GetObject(
                typeof(CoTempConv), "tcp://127.0.0.1:4000/TempConvCompDotNet");
            double choice, input, output;
            bool next = true;
            while (next)
            {
                Console.WriteLine("Please enter your choice:");
                Console.WriteLine("\t\t 1 - Converter from F to C");
                Console.WriteLine("\t\t 2 - From C to F");
                Console.WriteLine("\t\t 3 - exit");
                choice = Double.Parse(Console.ReadLine());
                if (choice == 1)
                {
                    Console.WriteLine("Input temperature in F: ");
                    input = Double.Parse(Console.ReadLine());
                    output = myCSTempConvComp.fToC(input);
                    Console.WriteLine(output);
                }
            }
        }
    }
}
```

```
        else if (choice == 2)
        {
            Console.WriteLine("Input temperature in C:");
            input = Double.Parse(Console.ReadLine());
            output = myCSTempConvComp.cToF(input);
            Console.WriteLine(output);
        }
        else
        {
            next = false;
            Console.WriteLine("See you next time.");
        }
    }
}
catch (Exception e)
{
    Console.WriteLine(e.ToString());
}
}
```

| Ví dụ

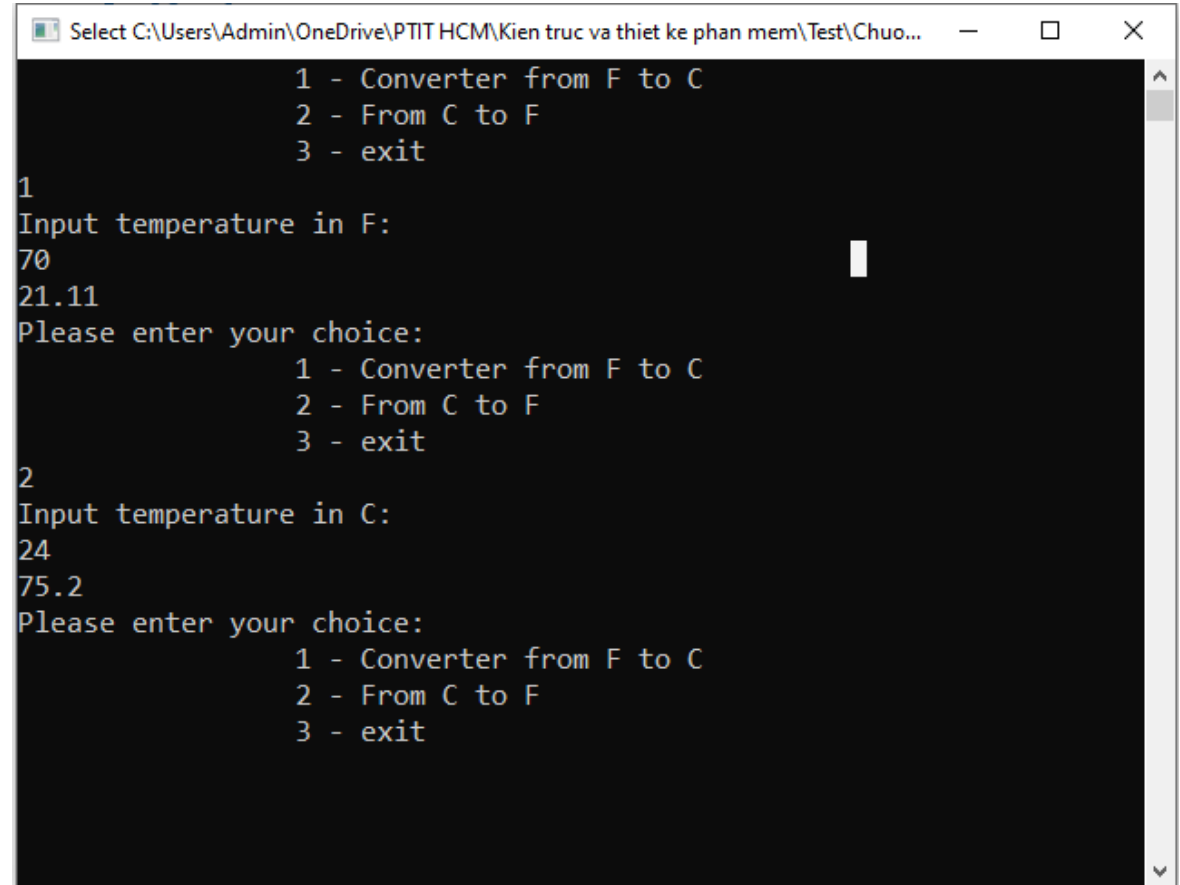
- Biên dịch:
 - `>csc CoTempConv.cs`
 - `>csc /r:CoTempConv.exe CoTempClient.cs`
- Chạy 2 file sau:
 - `CoTempConv.exe` -> Server component
 - `CoTempClient.exe` -> Client component
- Client tham chiếu tới thành phần từ xa bằng `Activator.GetObject()` và gọi phương thức của thành phần từ xa này.

| Kết quả



A screenshot of a terminal window titled "C:\Users\Admin\OneDrive\PTIT HCM\Kien truc va thiet ke phan mem\Test\Chuong 4...". The terminal displays the text "Hit <enter> to exit..." on a black background.

CoTempConv.exe



A screenshot of a terminal window titled "Select C:\Users\Admin\OneDrive\PTIT HCM\Kien truc va thiet ke phan mem\Test\Chuo...". The terminal shows a menu-driven interface for temperature conversion. It starts with a menu: "1 - Converter from F to C", "2 - From C to F", and "3 - exit". The user enters "1", followed by "Input temperature in F:", then "70". The result "21.11" is displayed. Then, the user enters "2", followed by "Input temperature in C:", then "24". The result "75.2" is displayed. Finally, the user enters "3" to exit. The menu is repeated after each conversion.

CoTempClient.exe

| 4.3.3 Gọi bất đồng bộ từ xa giữa các thành phần phân tán .NET

- Lập trình đồng bộ (**synchronous**) là cách lập trình mà các hoạt động của chương trình sẽ được thực hiện tuần tự.
- Lập trình bất đồng bộ (**asynchronous**) là cách lập trình cho phép các hoạt động thực hiện không theo tuần tự. Có thể các đoạn code ở dưới chạy trước đoạn code viết ở phía trên (bất đồng bộ), các hoạt động không phải đợi lẫn nhau.
- Gọi bất đồng bộ từ xa trong .NET dựa trên **Remoting Delegate**. Nó sẽ không khóa client trong khi chờ thông báo từ các thành phần từ xa.

I Ví dụ

Sử dụng lại CoTempConv, xây dựng Client bất đồng bộ:

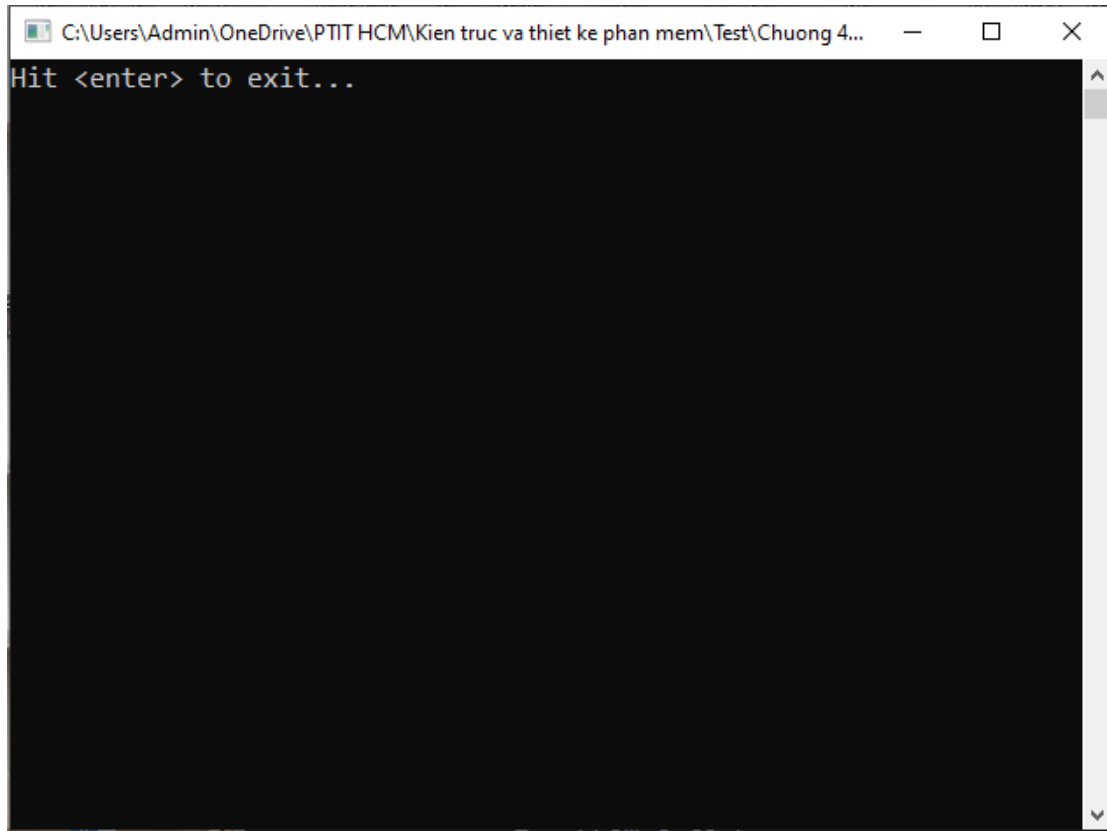
```
using System;
using System.Threading;
using System.Runtime.Remoting;
using System.Runtime.Remoting.Messaging;
using System.Runtime.Remoting.Channels;
using System.Runtime.Remoting.Channels.Tcp;

public class CoAsyncClient {
    public delegate double MyDelegate(double c);
    public static int Main(string [] args)
    {
        TcpChannel chan = new TcpChannel();
        ChannelServices.RegisterChannel(chan);
        CoTempConv obj =
            (CoTempConv)Activator.GetObject(typeof(CoTempConv),
            "tcp://localhost:4000/TempConvCompDotNet");
        if(obj == null)
            System.Console.WriteLine("Could not locate server");
        else {
            AsyncCallback cb = new AsyncCallback(CoAsyncClient.MyCallBack);
            MyDelegate d = new MyDelegate(obj.cToF);
            IAsyncResult ar = d.BeginInvoke(32, cb, null);
        }
        System.Console.WriteLine("Hit <enter> to exit ...");
        System.Console.ReadLine();
        return 0;
    }
    public static void MyCallBack(IAsyncResult ar)
    {
        MyDelegate d = (MyDelegate)((AsyncResult)ar).AsyncDelegate;
        System.Console.WriteLine(d.EndInvoke(ar));
        System.Console.WriteLine("See you next time");
    }
}
```

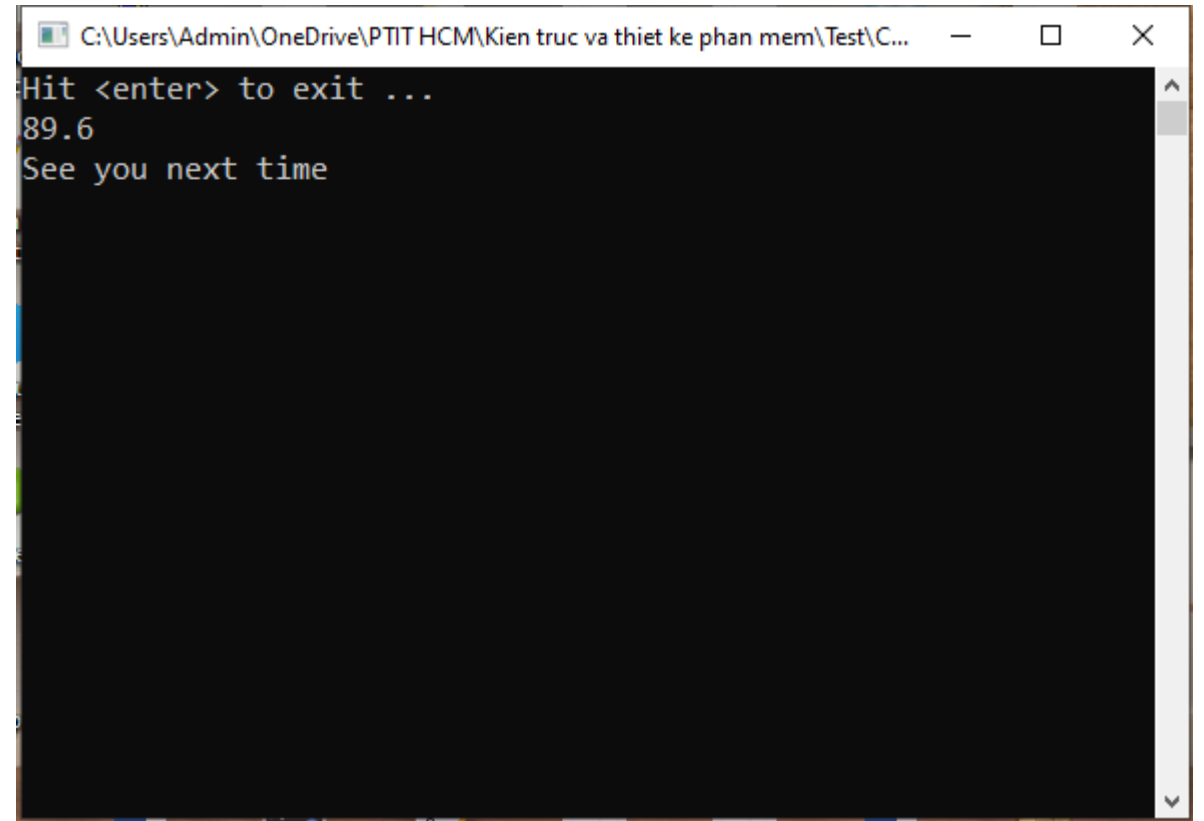
- Có hai Delegate trong ví dụ:
 - Một là **MyDelegate** trỏ tới phương thức từ xa **cToF** của thành phần từ xa có tên **TempConvDotNET**.
 - Delegate không đồng bộ khác là **AsyncCallback**, được truyền vào phương thức **BeginInvoke** của **MyDelegate**.

| Ví dụ

- Biên dịch `csc /r:CoTempConv.exe CoAsyncClient.cs`
- Chạy 2 file `CoTempConv.exe` và `CoAsyncClient.exe`.

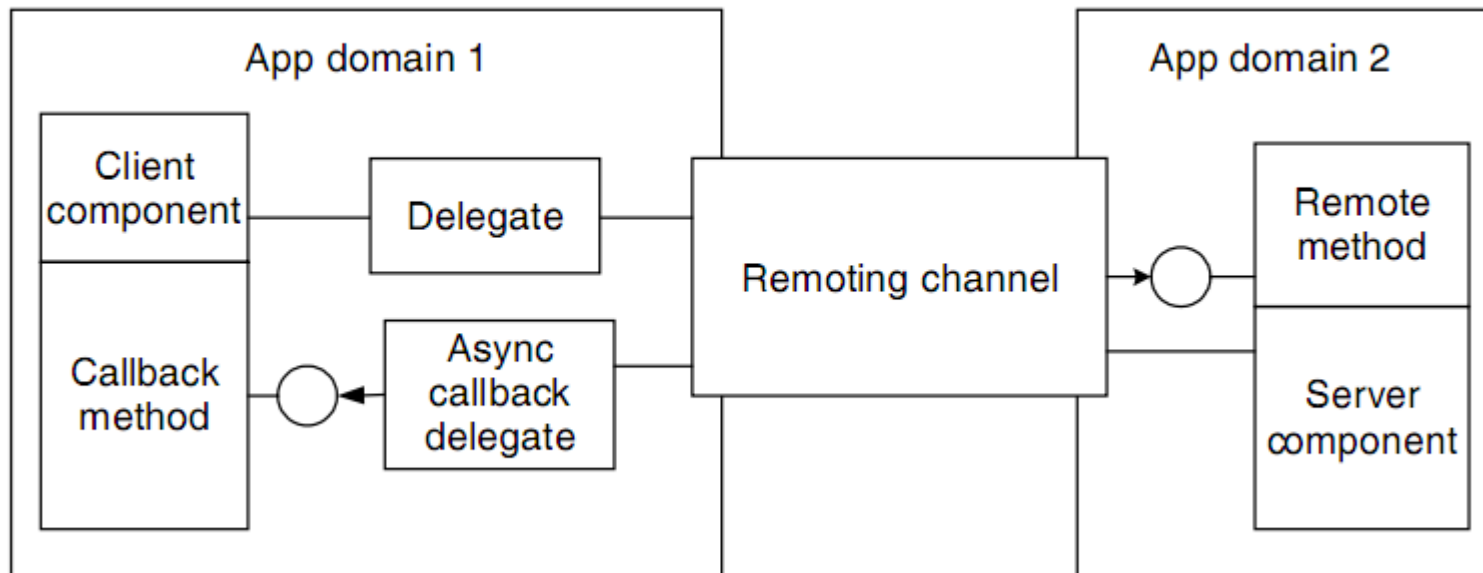
A screenshot of a Windows command prompt window titled "C:\Users\Admin\OneDrive\PTIT HCM\Kien truc va thiet ke phan mem\Test\Chuong 4...". The window contains the text "Hit <enter> to exit..." in a monospaced font.

CoTempConv.exe

A screenshot of a Windows command prompt window titled "C:\Users\Admin\OneDrive\PTIT HCM\Kien truc va thiet ke phan mem\Test\C...". The window contains the text "Hit <enter> to exit ...", "89.6", and "See you next time" in a monospaced font.

CoAsyncClient.exe

| Gọi bất đồng bộ từ xa giữa các thành phần phân tán .NET



| 4.4 MÔ HÌNH TRIỂN KHAI .NET

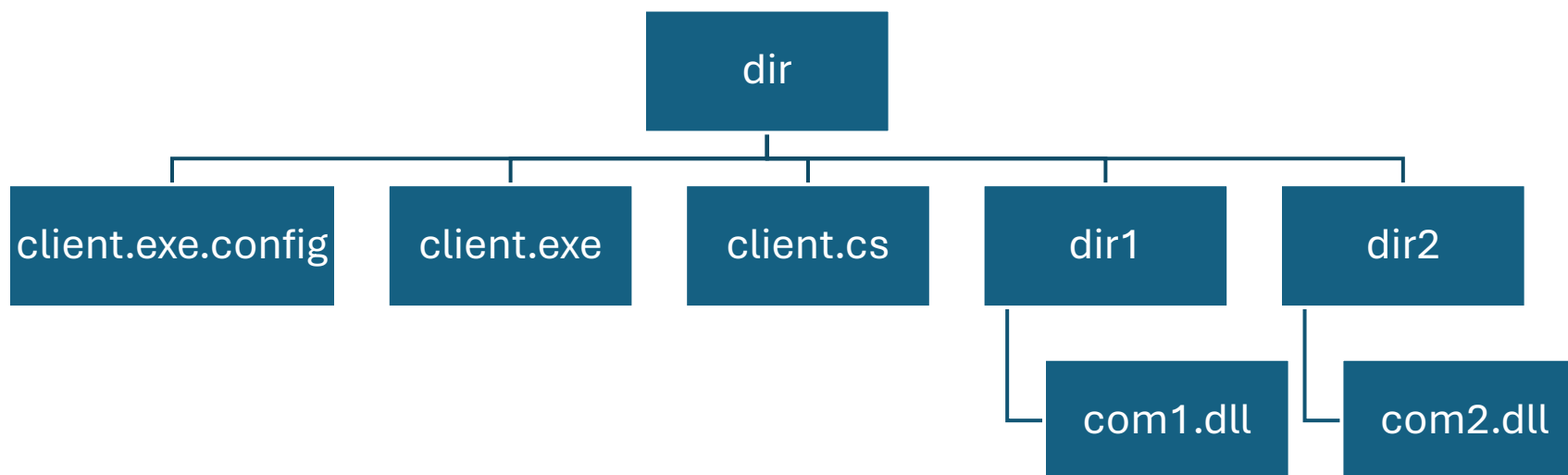
- Thành phần .NET có thể được triển khai như một thành phần riêng hoặc thành phần chia sẻ chung trong một file assembly.
- Assembly là một đơn vị triển khai cơ bản trong .NET.
- Thành phần riêng là thành phần trong đó nó sẽ được plug-in.
- Thành phần chia sẻ chung không biết thành phần nào sẽ sử dụng nó mà phải được đăng ký trong Global Assembly Cache (GAC). Thành phần chia sẻ hỗ trợ nhiều phiên bản thực thi.

| 4.1.1 Triển khai riêng

- Thành phần riêng được triển khai trên một thư mục các client hoặc thư mục con các client.
- Đây là triển khai đơn giản nhất được thực hiện bằng cách copy tất cả các thành phần tới nơi mà client cư trú.
- Gỡ bỏ việc triển khai được thực hiện bằng cách xóa bỏ mọi thành phần **.dll** trong thư mục.
- Thành phần riêng không hỗ trợ kiểm soát việc xác định phiên bản và chỉ triển khai trong nội bộ của một công ty.
- Cần có một file miêu tả cấu hình XML nếu các thành phần không được đặt trong cùng thư mục với client của nó. Đường dẫn riêng phải được xác định trong thẻ con **probing** của thẻ **assembly** building trong file cấu hình ứng dụng với phần mở rộng là “config.”

| Triển khai riêng

- Ví dụ:
 - >csc /t:library /out:dir1\comp1.dll comp1.cs
 - >csc /t:library /out:dir2\comp2.dll comp2.cs
 - >csc /r:dir1\comp1.dll, dir2\comp2.dll /out:client.exe client.cs



| Ví dụ file cấu hình

```
<configuration>
  <runtime>
    <assemblyBinding>
      <probing privatePath = "dir1; dir2"/>
    </assemblyBinding>
  </runtime>
</configuration>
```

- CLR sẽ sử dụng đường dẫn riêng để tìm kiếm thành phần cần thiết để tải nếu nó không tìm thấy thành phần cần thiết trong thư mục hiện thời.

| 4.4.2 Triển khai chia sẻ chung

- Cách triển khai thành phần có khả năng dùng lại phổ biến nhất là triển khai thành phần đăng ký với GAC.
- **Global Assembly Cache (GAC)** là nơi lưu trữ các .NET assemblies để có thể chia sẻ với các ứng dụng trong hệ thống.
- Thành phần chia sẻ có thể là duy nhất được xác định bởi cặp khóa public/private.
- Thành phần chia sẻ đã đăng ký với GAC có thể được chia sẻ ở mọi nơi.

| Các bước cần để tạo ra một thành phần .NET chia sẻ

- **Bước 1:** Tạo một cặp khóa public/private bằng cách sử dụng sn.exe.

>sn -k mykey.snk

- Khóa public dùng để xác minh lại khóa private.
- Khóa private được thể hiện như là một chữ ký số được lưu trong thành phần assembly.
- Khóa public được lưu trong file manifest của assembly.
- Khi client tham chiếu đến thành phần, mã thông báo của khóa public được lưu trong assembly của client.

| Các bước cần để tạo ra một thành phần .NET chia sẻ

- **Bước 2:** Nhúng các dòng sau vào mã nguồn của thành phần:

```
using System;  
using System.Reflection;  
[assembly:AssemblyKeyFile("mykey.snk")]  
[assembly:AssemblyDelaySign(false)]  
[assembly:AssemblyVersion("1.0.0.0")]
```

- Lệnh sau sẽ gán chữ ký vào thành phần ngay lập tức
>csc /t:library my_thành_phần.cs
- Lệnh tiếp theo sẽ lưu mã thông báo của khóa public vào thành phần của client
>csc /r:my_thành_phần.dll /out:myapplication.exe myapplication.cs
- Nếu cần trì hoãn việc gán chữ ký, chúng ta có thể gán chữ ký sau bằng cách dùng lệnh:
>sn -R my_thành_phần.dll mykey.sn

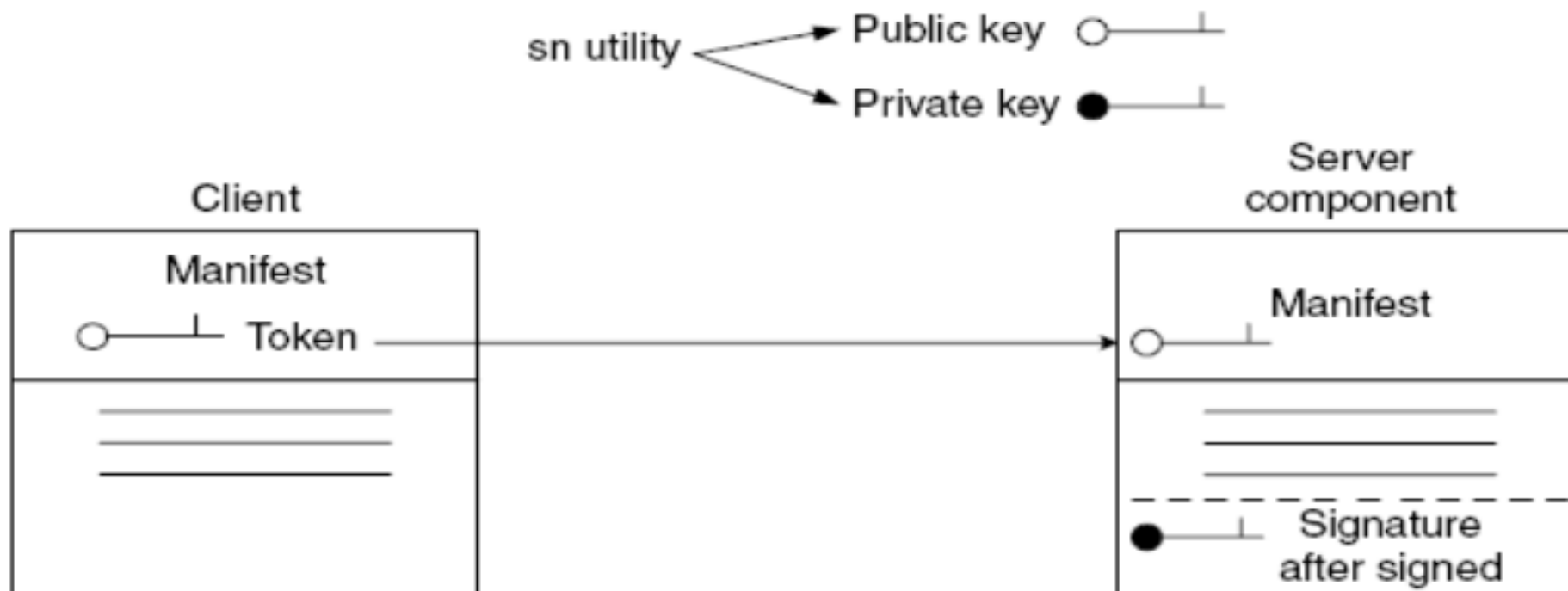
| Các bước cần để tạo ra một thành phần .NET chia sẻ

- **Bước 3:** Đăng ký thành phần chia sẻ với GAC

>gacutil /I my_thành_phần.dll

- Chữ ký được xác minh khi thành phần được đăng ký với GAC trong bước 3 để đảm bảo rằng thành phần không bị thay đổi trong khi assembly được xây dựng.
- Trong khi chạy, mã thông báo của khóa public trong manifest của client được xác minh với khóa public của định danh thành phần. Nếu chúng tương xứng với nhau thì nghĩa là đây chính là thành phần mong muốn.

| Cặp khóa public/private trong thành phần .NET



I Quản lý phiên bản trong GAC

- Số phiên bản của thành phần chia sẻ được đánh dấu bởi bốn số khác nhau và cách nhau bởi dấu chấm theo định dạng **major.minor.build.revision**, ví dụ 1.2.1.0.
- Việc thực thi được cài đặt bằng cách xác định phiên bản của .NET. .NET framework cho phép các thành phần có phiên bản khác nhau có cùng tên chạy trên các ứng dụng khác nhau.
- CLR kiểm tra các số **major** và **minor** trước tiên. Theo mặc định, chỉ cho phép phần **major.minor** hợp lệ. Số build theo mặc định là số tương thích ngược. Ví dụ, phiên bản 1.2.3.0 tương thích với 1.2.0.0 nhưng 1.2.3.0 không tương thích với 1.2.4.0.
- Nếu số phiên bản trong manifest của thành phần không tương thích với thành phần nào trong GAC, nó sẽ nạp một thành phần có phiên bản khác về revision, cũng có thể tùy biến trong file cấu hình ví dụ sau sẽ load 2.0.0.0 thay vì 1.2.3.4

```
<assemblyIdentity>  
  <binding Redirect oldVersion = "1.2.3.4" newVersion =  
  "2.0.0.0"/>  
</assemblyIdentity>
```

| Các bước cần để tạo ra một thành phần .NET chia sẻ

- **Bước 4:** Sử dụng thành phần chia sẻ.
- Client phải tạo tham chiếu đến thành phần chia sẻ để được sử dụng:

```
>csc /t:exe /r:my_thành_phần.dll /out:myapp.exe myapp.cs
```
- Để sử dụng lại thành phần chia sẻ, mã nguồn client phải sử dụng không gian tên có sẵn trong assembly.

Ví dụ triển khai chia sẻ TempConvComp

```
using System;
using System.Reflection;
[assembly: AssemblyVersion("1.0.0.0")]
[assembly: AssemblyKeyFile("originator.key")]
namespace TempConv
{
    public class TempConvComp
    {
        public TempConvComp()
        { }
        public double cToF(double c)
        {
            // how to control output format
            return (int)((c*9/5.0+32)*100)/100.0;
        }
        public double fToC(double f)
        {
            // how to control output format
            return (int)((f-32)*5/9.0*100)/100.0;
        }
    }
}
```

TempConvComp.cs

- Các bước sau xây dựng thành phần chia sẻ:

```
>sn -k originator.key
>csc /t:library /out:TempConv.dll TempConvComp.cs
>gacutil /i TempConv.dll
>csc /r:TempConv.dll /t:exe
/out:TempConvCSClient.exe TempConvCSClient.cs
```

Mã nguồn TempConvCSClient.cs
giống với ví dụ trước

Lưu ý:

- Chạy cmd bằng quyền Administrator khi chạy lệnh **gacutil**
- Add đường dẫn sau vào biến PATH trong Environment Variables để thực thi câu lệnh **sn**:
C:\Program Files (x86)\Microsoft SDKs\Windows\v10.0A\bin\NETFX 4.8 Tools
(hoặc đường dẫn .NET tương ứng trên máy tính)

Ví dụ triển khai chia sẻ TempConvComp

Test > Chương 4 > Triển khai thành phần chia sẻ				
Name	Date modified	Type	Size	
originator.key	22/03/2024 11:28 AM	KEY File	1 KB	
TempConv.dll	22/03/2024 11:29 AM	Application exten...	4 KB	
TempConvComp.cs	22/03/2024 11:13 AM	Visual C# Source F...	1 KB	
TempConvCSClient.cs	20/03/2024 11:01 AM	Visual C# Source F...	2 KB	
TempConvCSClient.exe	22/03/2024 11:31 AM	Application	5 KB	

Danh sách các files

```
D:\Test\Chương 4\Triển khai thành phần chia sẻ\TempConvCSClient.exe
Please enter your choice:
    1 - Converter from F to C
    2 - From C to F
    3 - exit
1
Please tell me the temperature in F:
70
result = 21.11
Please enter your choice:
    1 - Converter from F to C
    2 - From C to F
    3 - exit
2
Please tell me the temperature in C:
21
result = 69.8
Please enter your choice:
    1 - Converter from F to C
    2 - From C to F
    3 - exit
```

Kết quả chạy file TempConvCSClient.exe

```
Administrator: Command Prompt
D:\Test\Chương 4\Triển khai thành phần chia sẻ>gacutil /i TempConv.dll
Microsoft (R) .NET Global Assembly Cache Utility. Version 4.0.30319.0
Copyright (c) Microsoft Corporation. All rights reserved.

Assembly successfully added to the cache

D:\Test\Chương 4\Triển khai thành phần chia sẻ>csc /r:TempConv.dll /t:exe /out:TempConv
CSClient.exe TempConvCSClient.cs
Microsoft (R) Visual C# Compiler version 4.8.9037.0
for C# 5
Copyright (C) Microsoft Corporation. All rights reserved.

This compiler is provided as part of the Microsoft (R) .NET Framework, but only support
s language versions up to C# 5, which is no longer the latest version. For compilers th
at support newer versions of the C# programming language, see http://go.microsoft.com/f
wlink/?LinkID=533240

D:\Test\Chương 4\Triển khai thành phần chia sẻ>
```

Kết quả các câu lệnh chia sẻ

| Bài tập

- Phân biệt tham chiếu và tham trị trong .NET.
- Phân biệt các mô hình thành phần trong .NET.
- So sánh mô hình thành phần của .NET và J2EE.
- Viết chương trình chuyển đổi nhiệt độ sử dụng thành phần .NET.
- Viết chương trình chuyển đổi tỷ giá tiền sử dụng thành phần .NET.

| BÀI TẬP NHÓM – TÍNH ĐIỂM BÀI TẬP

Quy định:

- Nhóm từ 1 - 3 bạn.
- Nội dung: Mô hình hóa với UML và Xây dựng module chức năng với .NET.
- Hạn nộp bài: 06/04/2024 (In, đóng cuốn)
- Ghi chú: Nhóm chọn làm bài tập chương 3 hoặc chương 4 để lấy điểm bài tập.

Nhóm chọn 1 trong các Bài tập sau hoặc Có thể chọn 1 Hệ thống khác:

- Xây dựng module quản lý mua hàng trong Hệ quản lý bán sách online với mô hình .NET
- Xây dựng module quản lý mượn sách trong Hệ quản lý thư viện với mô hình .NET
- Xây dựng module quản lý cho thuê xe ô tô với mô hình thành phần .NET
- Xây dựng module quản lý đăng ký học tín chỉ với mô hình thành phần .NET
- Xây dựng module quản lý nghe nhạc online với mô hình thành phần .NET
- Xây dựng module quản lý đăng ký tour du lịch online với mô hình .NET
- Xây dựng module quản lý đăng ký mua vé máy bay online với mô hình thành phần .NET.